EMBEDDED SYSTEM FINAL PROJECT REPORT
DEPARTMENT OF ELECTRICAL ENGINEERING
UNIVERSITAS INDONESIA

ETLE Speed Detector

GROUP 3

Yohana Indah Nathania Br Sihotang	**2406368946**
Nayla Pramesti Adhina	**2406368901**
Muhammad Naufal Gilardino	**2406450434**
Muhammad Agib Anugrah Pratama	**2406450415**

PREFACE

This technical report discusses the process of developing the final project in the Embedded Systems course at the Electrical Engineering Department, Universitas Indonesia. Titled ETLE Speed Detector, this system detects the speed of a moving object using an Arduino Uno (ATmega328P) microcontroller through AVR Assembly code. The development process of this project is conducted collaboratively by four students from Group 3, where the purpose of developing the project is to incorporate all eight modules from the Embedded Systems course; GPIO, USART, Arithmetic, Timer, External Interrupt, PWM and EEPROM, ADC, and I2C into a unified embedded system. The application of this ETLE system acts to simulate the speed of a moving object between two infrared sensors.

The format of this report is arranged in such a way that it will include background information about the project, implementation of the modules, tests, results and the conclusions that can be drawn from this project. We sincerely hope that this report reflects the depth of knowledge gained through this course on embedded systems. We would like to take this opportunity to extend our thanks to our lecturers and lab assistants who guided us through this entire project. We would also like to thank our lab assistant Bang Alex for all his assistance throughout the project.

Depok, May 17, 2026

Group 3

TABLE OF CONTENTS

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Safety on the roads is an essential issue all over the world, where speeding is one of the primary reasons for crashes. In most urban and semi-urban cities, speed limit regulations have been implemented; however, their implementation depends on manual observation that may not be consistent and requires considerable manpower. It is necessary to develop automated speed detection systems that can operate in real-time.

Embedded technology provides an efficient and effective method to address this issue. Platforms based on microcontrollers, like the Arduino Uno, which operates using the ATmega328P AVR microcontroller, possess adequate computing capabilities and I/O peripherals needed for building a stand-alone speed detection system. Integrating IR sensors for detecting vehicles, Timer1 for time measurements, and ADC for determining speed thresholds, a functional speed detector can be developed at a low cost.

A similar implementation will be carried out in this project as the end project for the Embedded Systems class. It involves designing a system to address all the eight lessons of the course, which include; General Purpose Input/Output, USART Serial Communication, Arithmetic Operations, Timer, External Interrupts, PWM & EEPROM, Analog Digital Converter Interface and Inter-Integrated Circuit communication, and all are implemented purely in the AVR Assembly Language.

In the real-world scenario that is used in this project, a toy car moving between two IR sensors on a particular track at Hotwheels scale. The amount of time the car takes to move through the track is recorded and analyzed to determine its speed, which is then compared with the preset user-defined value, which is adjustable by use of a potentiometer. Immediate visual output using an LED display and LCD and audible alerts using a buzzer are triggered when a particular speed is exceeded.

1.2 PROJECT DESCRIPTION

Project Title: ETLE Speed Detector

The name of the proposed project is 'ETLE Speed Detector', which is a speed measurement system of vehicles that work on the ETLE Technology. The project is developed using the Arduino Uno microcontroller board with all modules in AVR Assembly Language.

The working principle of the project is as follows: two IR sensor modules (FC-51) are placed at a fixed distance of 20 centimeters (0.2 meters) on a track. As soon as the toy car crosses the first sensor (IR Sensor 1), an interrupt is generated at INT0 and the Timer1 starts. Afterward, when the toy car passes IR Sensor 2, an interrupt occurs at INT1 and Timer1 stops. Time counted by the Timer1 with the distance between both sensors is used to determine the speed of the vehicle through the following formula:

$$Speed\ (km/h) = (Distance\ (m) \div Time\ (s)) \times 3.6$$

The detected speed is then compared with a user-configurable threshold speed. The threshold speed is read using a potentiometer input into analog pin A0 of the ATmega328P, ranging from 20-100km/h through its ADC module. There are three levels of outputs:

a. Green LED + LCD "SAFE SPEED" – when the vehicle speed falls between 10km/h and the threshold
b. Yellow LED + LCD "TOO SLOW" – when the vehicle speed is less than 10km/h
c. Red LED + LCD "OVER SPEED!" + Buzzer alarm – when the vehicle speed is greater than the user-configured threshold speed

The I2C LCD display (16×2 with a PCF8574 backpack) is controlled via TWI implementation in assembly code using TWCR, TWDR, and TWSR registers. The active buzzer output (overspeed alarm), using PWM in timer0, uses OC0A port pin PD6. Finally, the last detected overspeed value is saved in the EEPROM and shown by the LCD at startup in order to inform about the last overspeeding event. Speed information is also sent to a host computer via USART serial connection.

1.3 OBJECTIVES

The objectives of this project include the following:

a. The development and implementation of a real-time speed detection of vehicles through the use of two IR sensors and Timer1 of the ATmega328P microcontroller.

b. The use of all eight embedded systems modules in the design of this AVR Assembly project, which includes GPIO, USART, Arithmetic, Timer, Interrupt, PWM/EEPROM, ADC, and I2C.
c. Multi-sensor output, which includes visual (using LED and LCD) and audio (using buzzer) based on the detected speed classification.
d. The inclusion of user-programmable speed threshold limits using the potentiometer, which is connected to the ADC, with limits set between 20 km/h and 100 km/h.
e. The use of EEPROM for the permanent storage of overspeeding events even after power off.
f. Real-time transmission of speed data to a host computer using USART.

1.4 ROLES AND RESPONSIBILITIES

The roles and responsibilities assigned to the group members are as follows:

Roles	Responsibilities	Person
Firmware Developer & Report	AVR Assembly coding for GPIO, USART, Arithmetic, and Timer1 modules; PDF report writing	Yohana Indah Nathania Br Sihotang
Hardware Assembly & README	Circuit wiring and breadboard assembly; README documentation	Nayla Pramesti Adhina
Firmware Developer & Simulation	AVR Assembly coding for Interrupt, PWM, and EEPROM modules; Proteus circuit simulation	Muhammad Naufal Gilardino
Firmware Developer & Simulation	AVR Assembly coding for ADC and I2C modules; Proteus circuit simulation	Muhammad Agib Anugrah Pratama

Table 1. Roles and Responsibilities

CHAPTER 2
IMPLEMENTATION

2.1 EQUIPMENT

The roles and responsibilities assigned to the group members are as follows:

No.	*Component / Tool*	*Quantity*	*Purpose*
1	Arduino Uno (ATmega328P)	1 unit	Main microcontroller board
2	IR Sensor Module FC-51	2 units	Vehicle detection (Sensor 1 & Sensor 2)
3	I2C LCD 16×2 (PCF8574 backpack)	1 unit	Real-time speed and status display
4	Traffic Light LED Module (R/Y/G)	1 unit	Visual speed status indicator
5	Active Buzzer Module	1 unit	Overspeed alarm
6	Potentiometer	1 unit	Adjustable speed limit threshold via ADC
7	Breadboard	1 unit	Circuit prototyping
8	Jumper Wires (M-to-M, M-to-F)	As needed	Component interconnections
9	Arduino USB Cable	1 unit	Programming and serial monitoring
10	Toy Car (Hotwheels-scale)	1 unit	Test vehicle for speed measurement
11	Hotwheels launcher	1 unit	Test vehicle for speed measurement
12	Arduino IDE	Software	AVR Assembly firmware development

Table 2. Equipment List

2.2 IMPLEMENTATION

The implementation of the ETLE Speed Detector encompasses all eight modules within the Embedded Systems course. Below is an explanation of how each module was implemented.

a. Module 2 — GPIO Configuration

All pins involved in input/output operations are configured through the AVR registers without making use of any library provided by Arduino. The two outputs from the infrared sensors are configured as digital inputs by clearing their bits on DDRD. Traffic light LEDs which are configured as digital outputs include the Red LED (PD4), the Yellow LED (PD5), and the Green LED (PD6). The output pin for the buzzer is also configured as a digital output. PIND register is used to read the sensor state at idle times.

b. Module 3 – Serial Communication (USART)

   Initialization of USART0 involves loading the appropriate baud rate divisor into registers UBRR0H and UBRR0L and enabling transmission through UCSR0B as well as setting data format (8 bits per character and one stop bit) through UCSR0C. A custom subroutine polls for the availability of the transmitter (UDRE0 flag on UCSR0A) and then writes characters into the transmit buffer (UDR0). On each detection event, the calculated speed is transmitted to Serial Monitor as shown below: "Speed: XX km/h".

c. Module 4 – Arithmetic (Speed Calculation)

   The entire speed calculation is made in AVR Assembly language using arithmetic operations of 16-bit registers. Timer1 measures the time difference between the two sensor events in terms of timer ticks. These ticks are converted into milliseconds and seconds knowing the prescaler value and the clock frequency. Then, speed is calculated as the distance between sensors in meters divided by time in seconds, times 3.6 to obtain the speed in km/h. MUL, ADD, SUB and ADIW instructions are used to perform the intermediate calculations. Then, the result integer value in km/h is stored in a register pair.

d. Module 5 – Timer (Timer1 for Elapsed Time)

   Timer1 is a built-in 16-bit timer available in ATmega328P microcontroller. Timer1 measures the exact time period between two sensor events. Timer1 is set to Normal mode and a suitable prescaler value is selected according to the expected vehicle speed and the sensors' separation distance. As the INT0 interrupt signal is received, TCNT1H and TCNT1L are reset and the timer is started by writing a prescaler value in TCCR1B. Once the INT1 interrupt event occurs, the timer is stopped by resetting TCCR1B.

e. Module 6 – External Interrupts (INT0 & INT1)

External interrupts INT0 and INT1 are designed to identify the falling edge of IR sensor signals from pins PD2 and PD3 respectively. The bits controlling the interrupt sense of EICRA are set to trigger the interrupt at falling edge, and the interrupt enable bits of EIMSK register are set for INT0 and INT1. The ISR for INT0 is responsible for resetting and starting Timer1, whereas ISR for INT1 is responsible for stopping Timer1 and raising a flag for execution of speed calculation and output tasks in the main loop routine.

f. Module 7 — PWM & EEPROM

For the active buzzer, Timer0's Fast PWM operation is utilized in conjunction with the OC0A pin (PD6). In case of overspeed occurrence, TCCR0A and TCCR0B registers are set in such a way that the buzzer produces an audible square wave signal, whereby the duty cycle is controlled through OCR0A. The buzzer stops buzzing after a set time period by setting all Timer control registers to zero. The EEPROM stores the last overspeeded value, and whenever there is any overspeed situation, it is saved to a specific EEPROM address according to the standard EEAR/EEDR/EECR writing process. At the beginning of execution, the stored overspeed value is retrieved from the EEPROM.

g. Module 8 – ADC (Adjustable Speed Limit)

There is a speed limit potentiometer attached to ADC channel 0 (connected to pin A0 / PC0). This module involves the use of an ADC, whose configuration takes place through Assembly programming using AVcc reference and ADC channel 0 in ADMUX, and finally by activating the ADC using a suitable prescaler in ADCSRA. Conversion is carried out by setting the ADSC bit, while the 10-bit output is obtained from ADCH/L after setting the ADIF flag. An output in the form of a speed limit of 20 to 100 km/h range (from 0 to 1023) will be shown in the LCD display.

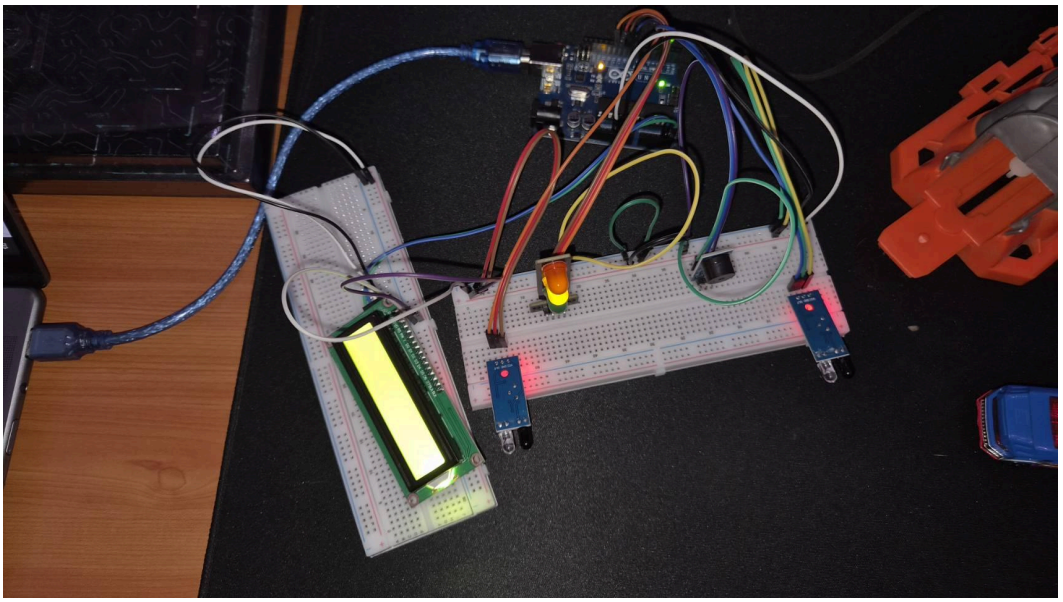
h. Module 9 – I2C / TWI (LCD Display)

The interface for the 16x2 LCD display with PCF8574 I2C Backpack is done using the Two-Wire Interface (TWI) available on the ATmega328P microcontroller, written in Assembly language. The TWI interface is initialized by setting up TWBR and enabling the peripheral through TWCR. Some lower-level functions that deal with sending out a START condition, sending out an address + write byte along with checking TWSR status code,

sending out data bytes, and finally, sending out the STOP condition are included. Higher-level LCD functions based on these subroutines take care of the nibble communication protocol needed to communicate with the PCF8574.

2.3 CIRCUIT DESIGN AND WIRING

The entire circuit is built on a breadboard using an Arduino Uno as the main control board. Two infrared sensor boards are mounted on the testing track with a gap of 20 centimeters between each other, with the OUT pins connected to D2 (INT0) and D3 (INT1) pins of the Arduino, respectively, and supplied power from the 5V and GND buses on the Arduino. The SDA and SCL pins of the I2C LCD are connected to A4 and A5 pins, respectively, of the Arduino, which are the TWI pins in the hardware configuration of the microcontroller. The Red, Yellow, and Green pins of the traffic light LEDs are connected to digital output pins (D11, D10, D9), while the pin for the active buzzer is connected to D8 pins.



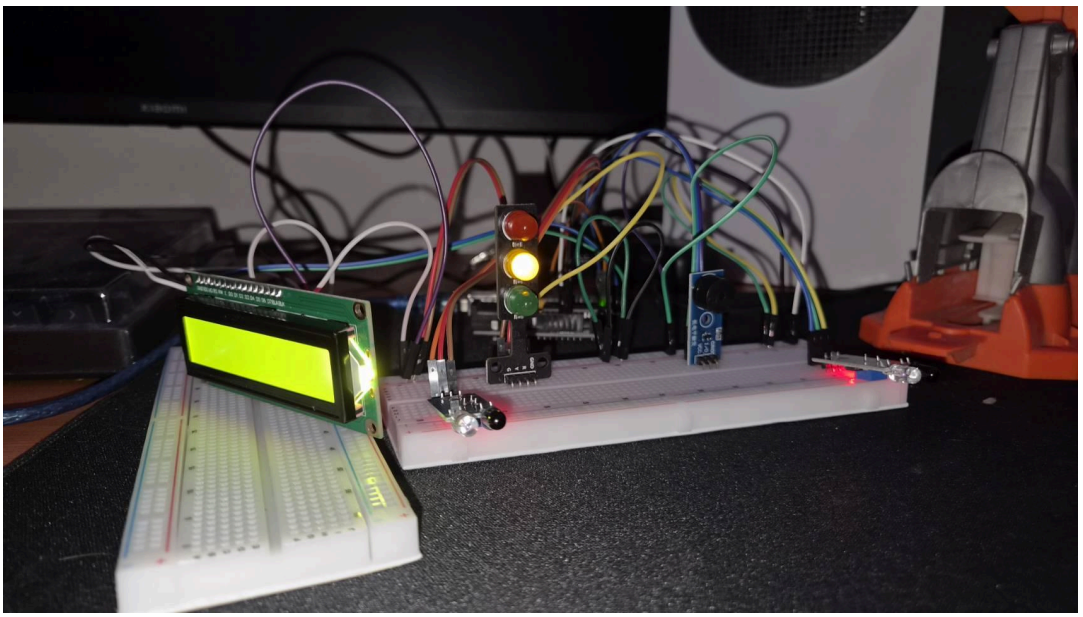


Fig 1. Hardware Circuit

CHAPTER 3
TESTING AND ANALYSIS

3.1 TESTING

Testing was performed to ascertain the correct functionality and integration of all the modules of the embedded system used in the Arduino Uno (ATmega328P) microcontroller, written in AVR Assembly, into the ETLE Speed Detector. The system was evaluated with a Hotwheels size toy car being driven down a track that had two infrared sensors, FC-51, attached. These sensors were set 20cm (0.2m) apart to capture the movement of the vehicle as accurately as possible.

The system operated as expected during testing on all parameters. The external interrupts (INT0 and INT1) were able to correctly trigger the start and stop functions of Timer1 as the toy car passed the first and second sensor and no interrupts were missed. The AVR Assembly arithmetic operations were able to correctly process the timer ticks to get the speed of the vehicle in km/h. In addition, the I2C LCD was constantly and accurately updating the actual speed and appropriate status messages (“SAFE SPEED” or “TOO SLOW”) based on these measurements. The adjustable speed threshold also worked as expected with the potentiometer linked to the ADC channel, and the target speed range (20-100km) could be varied by the testers during the test.

Most important, the buzzer module that was used was tested to verify it would not fail when used at high speeds. The system was able to successfully trigger overspeed protocols when the calculated speed of the toy car was greater than the user defined threshold. Timer0's Fast PWM operation was used to generate the square wave signal to be sent to the OC0A pin (PD6) and give a loud clear audible alarm sound from the buzzer. At the same time, the red traffic light LED came on, the LCD read “OVER SPEED!” and the event was successfully recorded in the EEPROM and stored permanently, thus ensuring the full and proper functioning of the alarm system.

3.2 RESULT

The results of the simulation showed that the ETLE Speed Detector was able to move through all programmed states as per the programmed speed limit of 5.0 km/h. When the program is run, the I2C LCD correctly shows “ETLE Detector Initializing…”

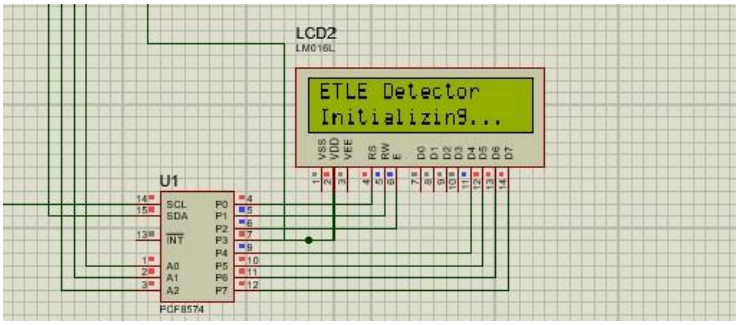


Fig. 2 ETLE Detector Opening

When the system was given a start command, it switched to its idle standby mode and displayed ‘Waiting…’ on the LCD and lit the yellow traffic LED, meaning the system was standing by waiting for a vehicle to activate the first infrared sensor,

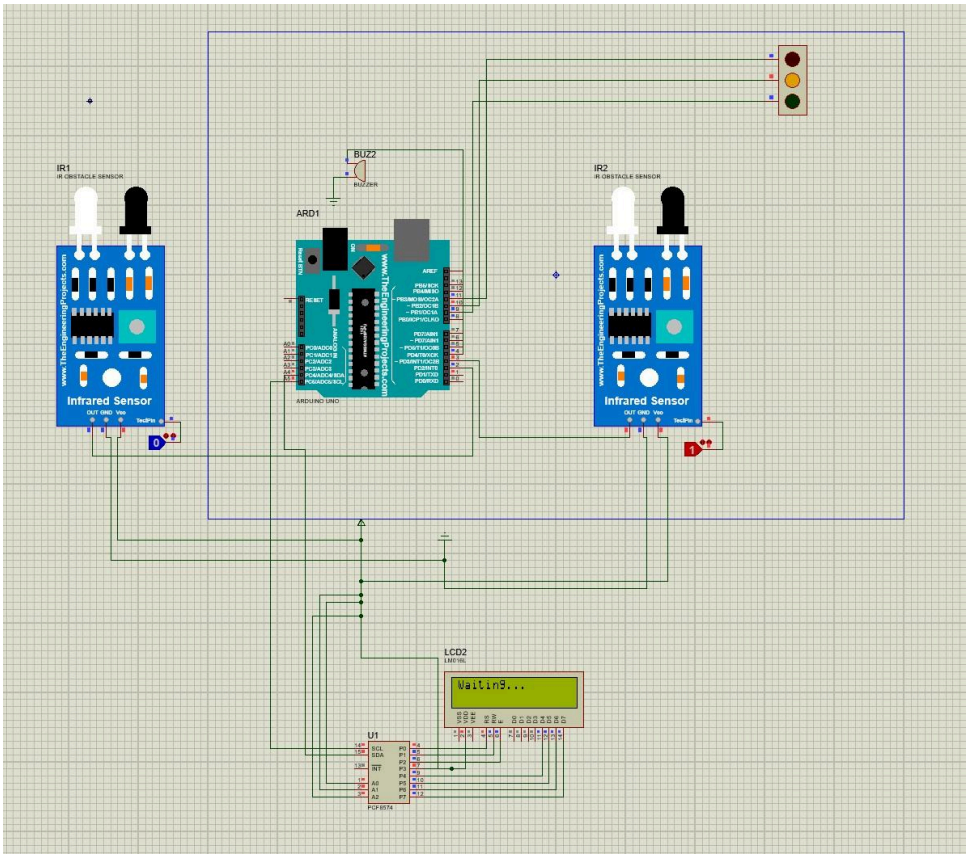


Fig. 3 ETLE Detector Idle State

Manual testing with logic states was carried out and the system was able to calculate the passing time, which is slow. The measured velocity was 0.3 km/h and the LCD was able to display “SAFE SPEED”. This was less than or equal to the 5.0 km/h limit and thus the system activated the green traffic LED (Fig. 1) as well. Since manually switching logic pins in Proteus is

not an accurate way to simulate the millisecond timing that is required for high-speed vehicles, a Virtual Terminal was used through USART communication to send simulated time intervals at a faster time than what is normal for Proteus. This test condition tested a simulated speed of 9.4 km/h. This was above the maximum of 5.0 km/h and the system correctly detected the overspeed and immediately displayed “OVER SPEED!”. The LCD displays “9.4 km/h” with the red traffic LED will light up and the buzzer will sound

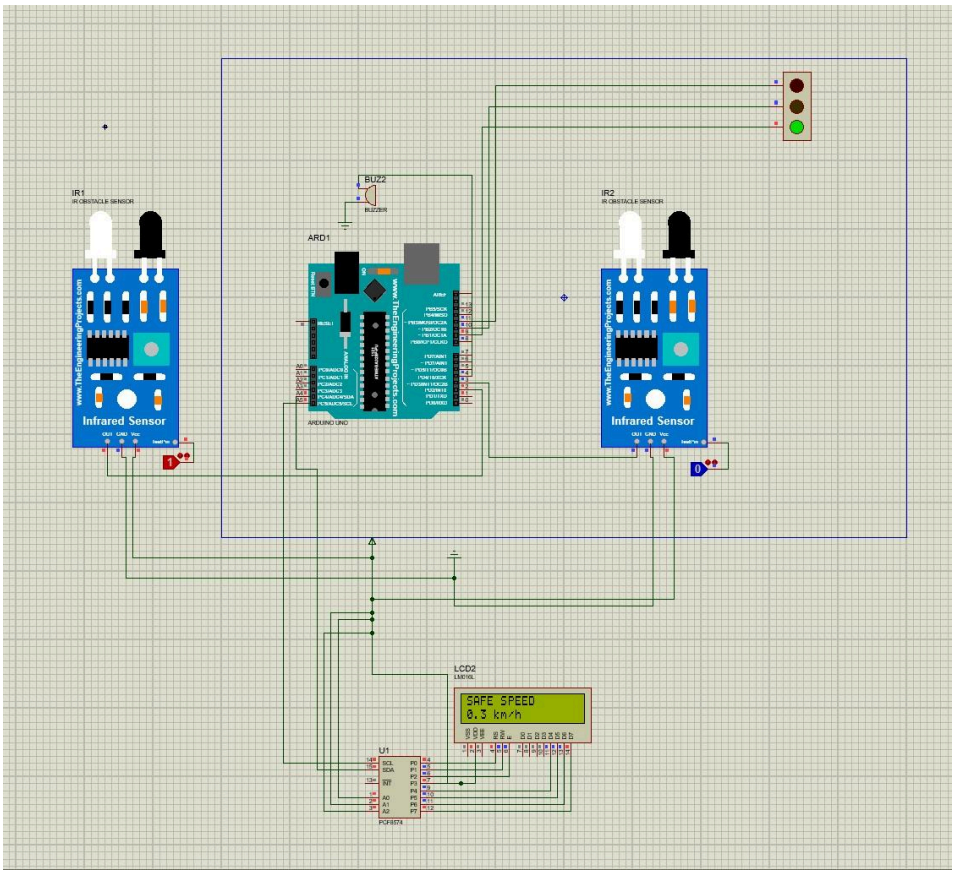


Fig 4. Testing Result Green Light (Safe Speed)

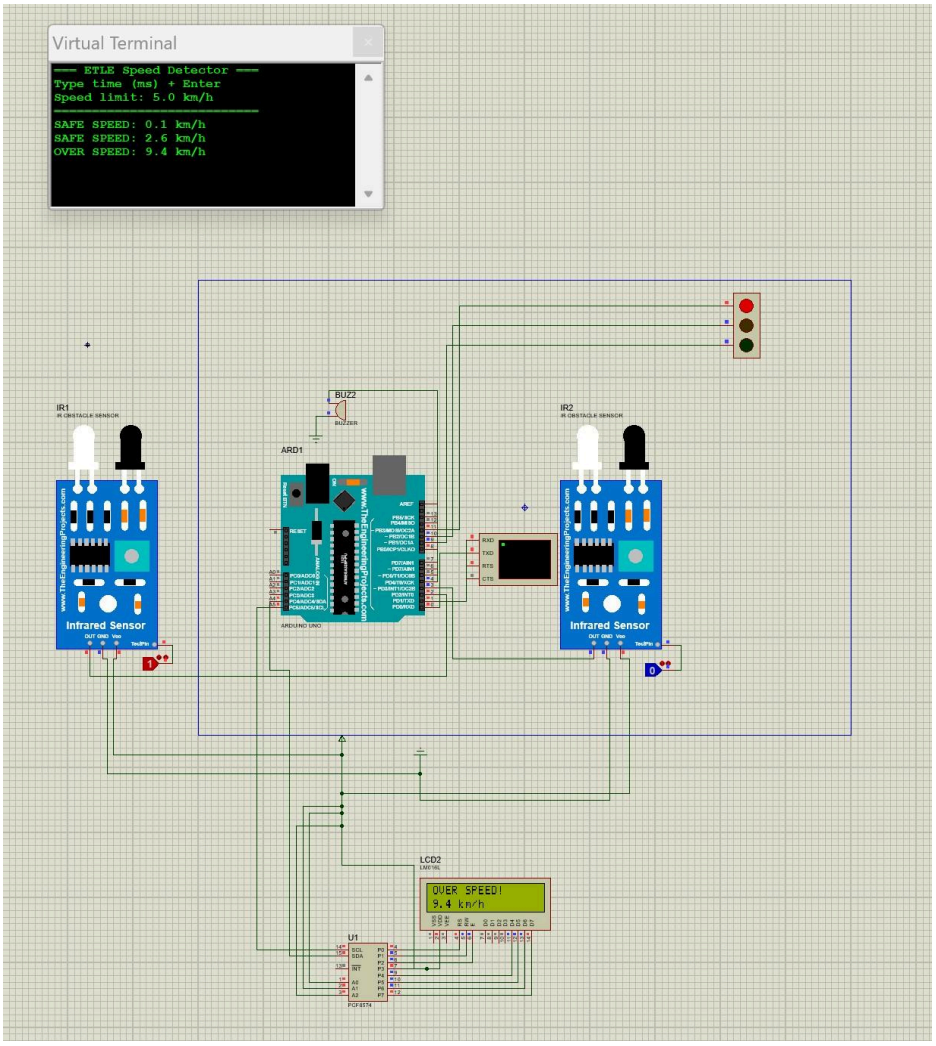


Fig 5. Testing Result Red Light (Over Speed)

After conducting all the simulations via software tools, the experiment in practical terms through the actual physical hardware was performed, and the outcome was extremely successful. All the integrated modules functioned as designed within the real-world conditions. The infrared sensors were able to detect the passing of the object, while the I2C LCD was successful in showing the speed of the object in real time. In order to test the actual functionality of the physical hardware in the practical scenario, the value of the overspeed limit was set to 1 km/h. When the overspeed limit is crossed, then the system activates the penalties correctly by turning on the red LED along with the audible buzzer.

3.3 ANALYSIS

The ETLE Speed Detector is successfully implemented which proves the very good integration of several low-level AVR Assembly modules, which comfortably surpasses the minimum requirement of 6 hardware/software features. Hardware timing is crucial to the

architecture and software polling is kept to a minimum. The 20cm interval between the triggers was then accurately measured by the 16-bit Timer1, which was triggered immediately by External Interrupts (INT0 and INT1) without CPU involvement. The project was able to introduce several communication protocols and data processing techniques. To drive the external LCD, I2C serial communication was used which reduced the number of external pins required by ATmega328P.

The data-processing algorithms in the system were correctly implemented in accordance with the branching logic defined in the ADC in the simulation with Proteus – the threshold was set to 5.0 km/h in software testing. The actuators behaved as expected: standard GPIO operation was used for controlling visual traffic light indicators (Yellow when no activity, Green when speed <= 5.0 km/hr, Red when speed > 5.0 km/hr); Timer0 was used to generate a Fast PWM signal to drive the auditory buzzer alarm during violation. In addition, the overspeed data is stored into EEPROM so that violation records are still stored even in the event of a power outage. Using a Virtual Terminal (USART) to circumvent the physical constraints of the Proteus simulation further shows a complete understanding of serial data transmission, and system debugging.

The software simulation was able to use a 5.0 km/h threshold, but because of mechanical issues that arose during testing, the maximum speed limit in the hardware implementation needed to be reduced to 1.0 km/h. The FC-51 infrared sensors required manual bending of the diodes to look across the track in order to accurately detect cars that are passing the sensor. Unfortunately, the components could not be bent to a perfect 90° angle so the sensor is slightly raised and may cause permanent damage to the structure or even break the sensor pins.

Thus, the standard low profile Hotwheels cars were not able to pass under the sensor-beams and would not be detected so a taller Hotwheels toy car had to be used. This taller vehicle was mechanically incompatible with the mechanical Hotwheels launcher, and so couldn't reach higher velocities during the test runs. To validate the overspeed branching logic, buzzer activation and red LED indicators under these mechanical constraints, the dynamic threshold was adjusted down to 1.0 km/h through the potentiometer. This adaptation was not only successfully physically tested, but has also proven the practical application and versatility of the system’s adjustable ADC input in real applications.

CHAPTER 4
CONCLUSION

ETLE Speed Detector was able to meet the target of creating an automated vehicle speed monitoring system. Using only AVR Assembly language to develop the firmware, this design was capable of handling all necessary hardware and time-based operations in an effective manner. In this way, all eight embedded systems modules were included in the design - GPIO, USART, Arithmetic, Timer, External Interrupts, PWM/EEPROM, ADC, and I2C - resulting in an efficient and effective safety application.

Timing was accurate using Hardware Interrupts (INT0 and INT1), along with 16-bit Timer1, to perform elapsed time calculations between the two infrared sensors without blocking CPU operations. Additionally, hardware tests showed that this design is flexible. Because of mechanical issues with the angle of the FC-51 sensors and the shape of the test cars, this project was able to decrease the speed limit up to 1.0 km/h thanks to the use of ADC to set an adjustable threshold value. With such a small threshold, this design correctly performed branching operations by sending Fast PWM pulses through a buzzer, setting the correct status of GPIO traffic light LEDs, transferring the information with USART protocol, and logging overspeeding violations in the EEPROM.

With regard to future developments, the mechanical design of the sensor mounts can be greatly enhanced using strong structural mounting or mechanical enclosures, which will enable the positioning of infrared diodes perfectly at 90 degrees without posing any harm to the component pins, making detection of regular profile cars accurate. Secondly, replacement of the infrared sensors by laser-break-beam sensors or ultrasonic sensors will expand detection ranges even under varying illumination. Lastly, inclusion of an automatic camera module that is initiated when there is a write-event on the EEPROM can further improve compatibility with existing ETLEs.

REFERENCES

[1] “Embedded System MBD,” DigiLab DTE. [Online]. Available: https://learn.digilabdte.com/books/embedded-system-mbd. [Accessed: May 17, 2026].

[2] Microchip Technology Inc., "ATmega48A/PA/88A/PA/168A/PA/328/P Datasheet," Microchip Technology, Chandler, AZ, USA, 2020. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf

[3] Arduino, "Arduino Uno Rev3 Documentation," *Arduino.cc*. [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3. (Accessed: May 17, 2026).

[4] Components101, "IR Sensor Module (FC-51) Pinout, Features & Datasheet," *Components101.com*, Sep. 2017. [Online]. Available: https://components101.com/sensors/ir-sensor-module. (Accessed: May 17, 2026).

[5] Texas Instruments, "PCF8574 Remote 8-Bit I/O Expander for I2C Bus," Texas Instruments, Dallas, TX, USA, Datasheet, Mar. 2015. [Online]. Available: https://www.ti.com/lit/ds/symlink/pcf8574.pdf

[6] NXP Semiconductors, "I2C-bus specification and user manual," NXP Semiconductors, Eindhoven, Netherlands, User Manual UM10204, Apr. 2014. [Online]. Available: https://www.nxp.com/docs/en/user-guide/UM10204.pdf

[7] MaxEmbedded, "AVR Timers – PWM Mode," *MaxEmbedded.com*, Aug. 2011. [Online]. Available: https://maxembedded.wordpress.com/2011/08/07/avr-timers-pwm-mode-part-i/. (Accessed: May 17, 2026).

[8] “ETLE Speed Detector with Arduino and Proteus,” YouTube. [Online Video]. Available: https://youtu.be/ILtYXEOpeeo?si=oAUPgLl04KOkYxWC. [Accessed: May 17, 2026].

APPENDICES

Appendix A: Project Schematic

ETLE-Group 3

https://drive.google.com/drive/folders/1tIVCrFV-kc9I0IGUbZujN52r8Y2C6OHk?usp=drive_link

Appendix B: Documentation